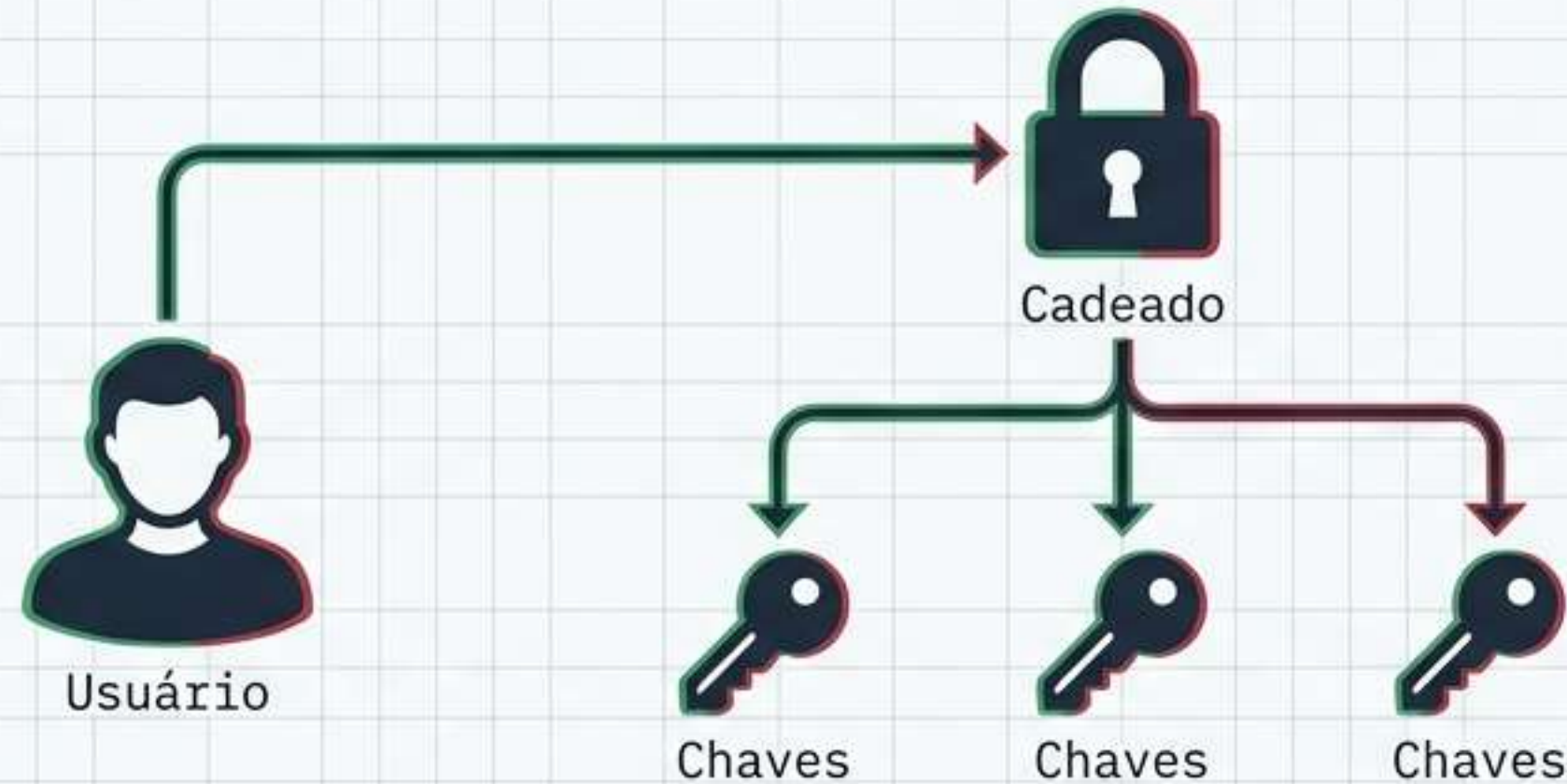




RBAC na Prática: Do Amador ao Profissional

Arquitetura, Banco de Dados e Código PHP para Controle de Acesso Baseado em Perfis.



Portão 1: Autenticação

Quem é você?

`login` / senha

O sistema descobre sua identidade.

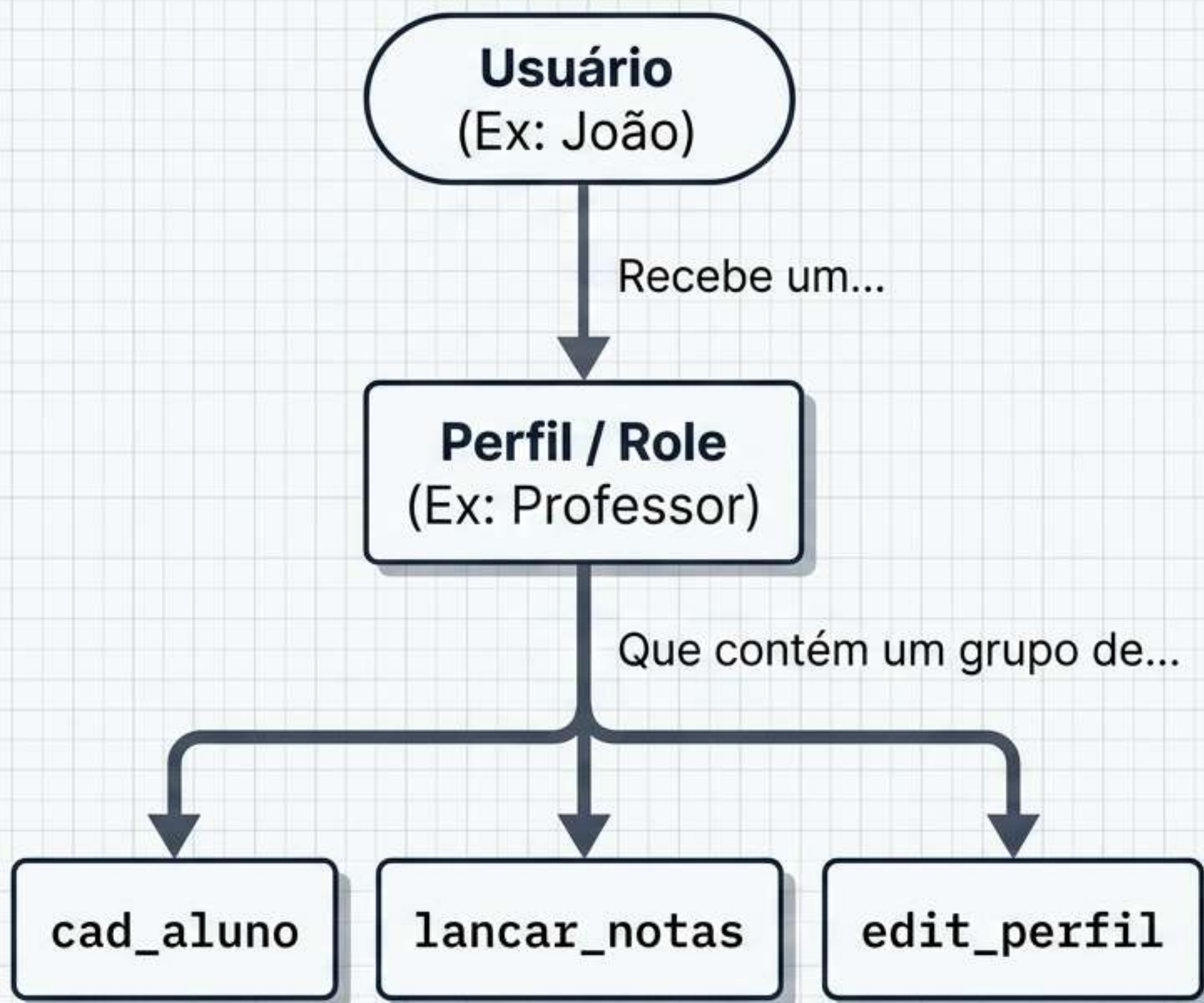


Portão 2: Autorização

O que você pode fazer?

O sistema verifica se o usuário autenticado tem permissão para a ação (ex: acessar financeiro, excluir aluno).

Todos entram pelo primeiro portão.
O RBAC controla o segundo.



O que é RBAC?

Em vez de dar permissões **diretamente** ao usuário (difícil **manutenção**), atribuímos **Perfis** (funções).

Cada perfil herda um conjunto de Permissões específicas e granulares.

RBAC na Prática (Controle de Estoque)

	prod_view	prod_add	prod_edit	prod_del
Balcão	✓	✗	✗	✗
Gerente	✓	✗	✓	✗
Dono	✓	✓	✓	✓

Se o Gerente for rebaixado para Balcão, mudamos apenas o perfil do usuário. As permissões se ajustam automaticamente.

A Transição Mental: Papéis vs. Ações

❌ Verificando o Perfil (Rígido)

```
if ($usuario->perfil == 'gerente') {  
    exibirBotaoEditar();  
}
```

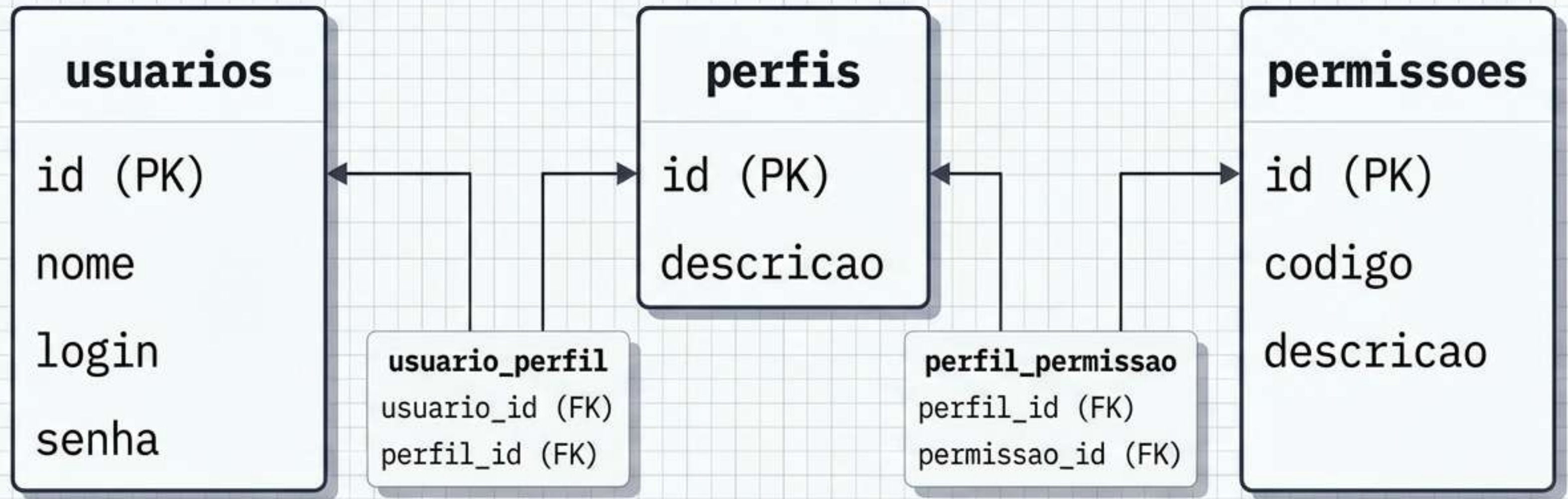
Se o 'Dono' também precisar editar, editar, o código quebra ou exige um OR infinito.

✅ Verificando a Permissão (Dinâmico)

```
if (hasPermission('prod_edit')) {  
    exibirBotaoEditar();  
}
```

O código front-end não quer saber quem é o usuário, apenas se ele possui a chave certa.

O Alicerce: Modelagem do Banco de Dados



Implementação PHP (1/3): A Sessão

```
// Após validar login e senha com sucesso...
$_SESSION['usuario_id'] = $user['id'];
$_SESSION['nome'] = $user['nome'];
$_SESSION['perfil'] = $user['perfil_descricao'];

// CARREGA AS PERMISSÕES NO EXATO MOMENTO DO LOGIN
// Evita múltiplas consultas ao BD durante a navegação
$_SESSION['permissoes'] = [
    'cad_aluno',
    'edit_aluno',
    'financeiro'
]; // Array gerado a partir de um JOIN nas 5 tabelas
```

Performance: Salvar os códigos de permissão em um array na sessão evita sobrecarregar o banco de dados a cada clique do usuário.

Implementação PHP (2/3): O Middleware

seguranca.php

```
function verificaPermissao($codigoRequerido) {  
    // Verifica se usuário está logado  
    if (!isset($_SESSION['usuario_id'])) {  
        header('Location: login.php');  
        exit;  
    }  
  
    // Verifica se o array possui a permissão específica  
    $permissoesUser = $_SESSION['permissoes'] ?? [];  
  
    if (!in_array($codigoRequerido, $permissoesUser)) {  
        header('Location: acesso_negado.php');  
        exit;  
    }  
}
```

O Coração do Sistema.

Este é o mesmo conceito utilizado em frameworks modernos (como Laravel ou Symfony). Centralizamos a segurança Symfony). Centralizamos a segurança em uma única função, mantendo o código modular e aderindo ao princípio DRY (Don't Repeat Yourself).

Implementação PHP (3/3): Protegendo a Aplicação

Proteção de UI (Usabilidade)

Home | Alunos | **Financeiro**

```
if(in_array('financeiro', $_SESSION['permissoes'])) {  
    echo '<li>Financeiro</li>';  
}
```

Esconder opções do menu melhora a navegação, mas não é segurança real.

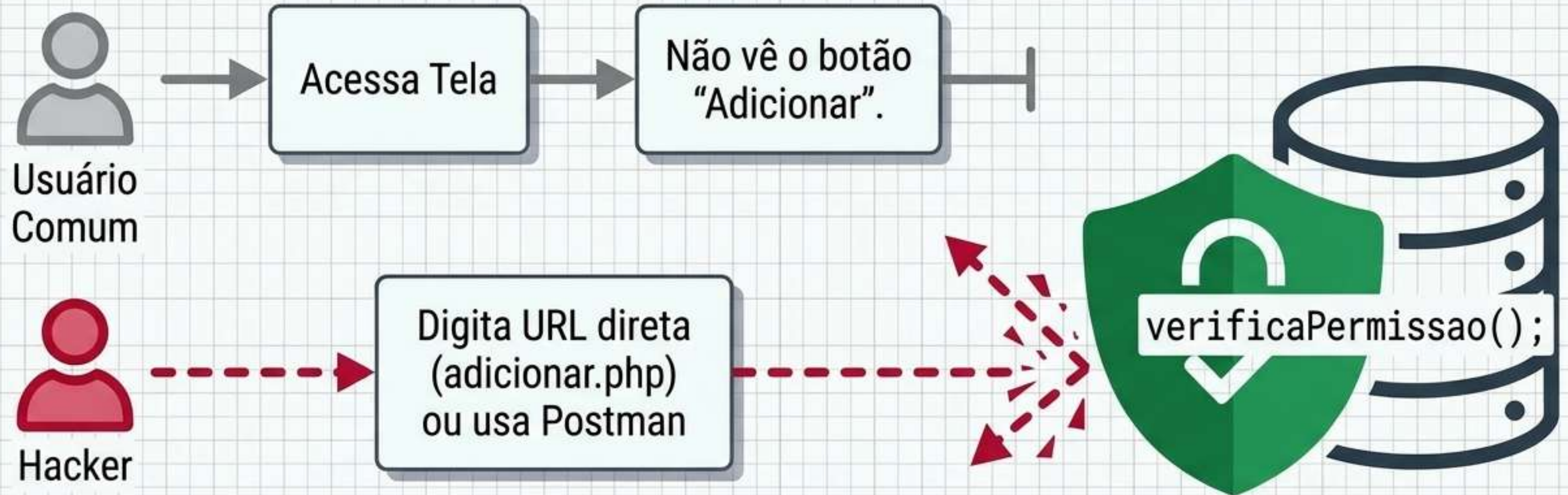
Proteção de Back-end (Segurança Real)

financeiro.php

```
require_once 'includes/seguranca.php';  
verificaPermissao('financeiro'); // Bloqueia a execução aqui!
```

A página alvo deve executar a trava de segurança antes de processar qualquer HTML ou consulta ao banco de dados.

O Paradoxo do Front-End: Ilusão vs. Segurança



**Seja incisivo nas validações de Ações no back-end.
Esconder a porta não adianta se ela continuar destrancada.**

Nível Avançado: Autorização em Nível de Dados



Professor João

Permissão: **lancar_notas**

Acesso à página concedido.
Mas quais notas ele pode ver?

❌ O Erro (Acesso Global)

```
SELECT *  
FROM turmas;
```

Resultado: João acessa as notas das turmas da escola inteira, corrompendo a integridade do sistema.

✅ O Correto (Acesso Escopado)

```
SELECT *  
FROM turmas  
WHERE professor_id = ?;
```

Resultado: João acessa apenas as notas de suas próprias turmas.

O RBAC libera a tela. O SQL restringe os dados pertencentes ao usuário.

A Última Camada: Trilha de Auditoria (Logs)

Data/Hora	Usuário	Ação	IP
24/07/2026 - 10:15	Maria (ID: 8)	Acesso ao painel	192.168.1.50
24/07/2026 - 15:33	Robson (ID: 15)	✓ Alterou nota do aluno João	192.168.0.15
24/07/2026 - 16:01	Sistema	Backup automático	Localhost

Quem alterou? Quando? De Onde?

A auditoria é essencial para rastreabilidade de ações críticas (edições e exclusões) liberadas pelas permissões.

Em caso de falha ou má conduta, o Log conta a história exata do sistema.

O Checklist do Arquiteto Web



Autenticação Segura

Uso exclusivo de `password_hash()` e `password_verify()`.



Defesa de Sessão

Executar `session_regenerate_id(true)` após o login.



Proteção de Requisição

Validar toda ação no servidor e usar tokens contra CSRF.



Blindagem de Dados

Uso rígido de Prepared Statements contra SQL Injection.



Defesa Front-end

Escapar saídas HTML para evitar XSS.

Dominar essas camadas em PHP procedural cria a base arquitetural sólida necessária para dominar ecossistemas modernos e frameworks.